

Nome

Matr.

1. Il design pattern Bridge permette alla classe client di
  - (a) Non tenere riferimenti
  - (b) Tenere riferimenti a oggetti che sono sottotipi di Implementor
  - (c) Tenere alcuni riferimenti a oggetti sottotipi di Implementor
  - (d) Non dover chiamare i metodi di Implementor
2. Per il design pattern Observer, il Subject
  - (a) Invia il riferimento a se stesso
  - (b) Implementa un metodo update
  - (c) Chiama un metodo e non passa parametri a tale metodo
  - (d) E' una classe astratta o un'interfaccia
3. Quando si usa il Mediator
  - (a) Le classi sono più accoppiate
  - (b) Le classi sono più coese
  - (c) Una classe deve conoscere poche classi
  - (d) Una classe deve conoscere tante classi
4. Rispetto a programmare singolarmente, l'effetto del pair programming è
  - (a) Produrre adeguata documentazione
  - (b) Rallentare lo sviluppo del sistema
  - (c) Produrre meno codice
  - (d) Produrre più codice
5. La fase di progettazione produce e documenta
  - (a) Le specifiche del software
  - (b) Le modalità di utilizzo del software
  - (c) Le funzionalità del software
  - (d) La struttura del software che realizza le specifiche
6. Per il design pattern Composite, il ruolo Composite
  - (a) Chiama metodi su istanze di sottotipi di Component
  - (b) Deve conoscere il tipo della classe client
  - (c) Non deve conoscere più di un riferimento
  - (d) Deve conoscere il tipo dell'oggetto semplice
7. Il design pattern Facade
  - (a) Fa aumentare le linee di codice delle classi che chiamano operazioni sul Facade
  - (b) Semplifica le classi che chiamano operazioni sul Facade
  - (c) Fa sì che alcune classi diventino strettamente accoppiate
  - (d) Si usa quando si hanno funzionalità simili fra varie classi
8. La soluzione per il design pattern Object Adapter avrà
  - (a) Una istanza per ciascun ruolo Adapter, Adaptee, Target
  - (b) Solo una istanza per il ruolo Adapter
  - (c) Nessuna istanza
  - (d) Una istanza per ciascun ruolo Adapter e Adaptee
9. Quale delle seguenti è uno svantaggio dell'applicazione del pattern Prototype?
  - (a) Il client non conosce il tipo concreto di prodotto usato, quindi ne è indipendente
  - (b) Il prototipo deve essere istanziato
  - (c) Per gli oggetti da usare come prototipi devo implementare in più il metodo clone()
  - (d) Si implementa un flusso di esecuzione più difficile da comprendere
10. Il Singleton e le sue varianti indicano sempre
  - (a) Un'unica istanza di una certa classe
  - (b) Un numero di istanze pari a 2
  - (c) Un numero di istanze dipendenti dalla memoria disponibile
  - (d) Un numero di istanze deciso dalla classe Singleton
11. Per il design pattern Chain of Responsibility, il destinatario di una richiesta
  - (a) Non conosce il tipo a runtime successivo nella catena
  - (b) Conosce il tipo a runtime successivo nella catena
  - (c) Ha il riferimento dell'elemento che lo precede nella catena
  - (d) Ha informazioni sul numero di elementi della catena
12. Un vantaggio del design pattern State è
  - (a) Avere in un unico posto la logica che cambia lo stato
  - (b) Avere una classe che nasconde un sottosistema
  - (c) Ridurre il numero di classi che implementano un certo comportamento
  - (d) Eliminare invocazioni indirette tra il client e gli oggetti usati
13. La tecnica di refactoring Estrai Metodo si applica
  - (a) A classi che hanno tanti attributi e tanti metodi
  - (b) Principalmente ai metodi che hanno tante responsabilità
  - (c) Solo ai metodi lunghi
  - (d) Alle classi grandi
14. Una classe con tante linee di codice e tante responsabilità
  - (a) Serve ad avviare operazioni su altre classi
  - (b) Contribuisce a diminuire la complessità di altre classi
  - (c) Serve a creare istanze di oggetti
  - (d) È molto complessa
15. Nei diagrammi UML di interazione (o collaborazione)
  - (a) Le classi sono nodi di un grafo e le relazioni tra classi sono frecce
  - (b) Gli oggetti sono allineati nella prima riga in alto
  - (c) L'oggetto è un nodo di un grafo e le interazioni sono frecce
  - (d) L'interazione tra oggetti è una linea con sopra una freccia numerata

16. Per una certa implementazione, il design pattern Decorator
- (a) Può fornire una lista delle istanze di classi del pattern
  - (b) Richiede tipicamente tante classi client
  - (c) Richiede di implementare metodi con lo stesso nome su classi diverse
  - (d) Lascia che i nomi dei metodi pubblici siano scelti indipendentemente fra le classi

Nome

Matr.

1. Le relazioni dei diagrammi UML delle classi indicano
  - (a) I metodi chiamati da una certa classe
  - (b) Le classi usate da una certa classe
  - (c) Gli oggetti usati da un certo oggetto
  - (d) Gli oggetti usati da una certa classe
2. Un vantaggio del Mediator è
  - (a) Fornire un'unica semplice interfaccia
  - (b) Rendere un sistema fortemente connesso
  - (c) Aumentare la specializzazione delle classi
  - (d) Ridurre la specializzazione delle classi
3. Il riuso tramite ereditarietà può essere fatto quando
  - (a) Si comprende il codice della superclasse
  - (b) Si vogliono avere tante classi
  - (c) Si conosce il solo nome della superclasse
  - (d) Si vogliono avere metodi overloaded
4. Spaghetti code indica che il codice
  - (a) Deriva da buone regole di progettazione
  - (b) Deriva dalla progettazione ad oggetti
  - (c) Ha metodi lunghi
  - (d) Ha tante piccole operazioni
5. Uno dei vantaggi del design pattern Observer è
  - (a) Isolare alcune classi
  - (b) Ridurre il numero totale di chiamate di metodo
  - (c) Ridurre il numero di classi implementate
  - (d) Isolare il codice che chiama metodi di varie classi
6. È consigliabile usare il processo a cascata quando
  - (a) Si vuol informare il cliente con documenti precisi durante lo sviluppo
  - (b) Si vogliono produrre sistemi software di piccole o medie dimensioni
  - (c) Si vogliono produrre documenti precisi per requisiti, progettazione, etc.
  - (d) Si vuole adattare continuamente il prodotto alle richieste del cliente
7. La soluzione per il design pattern Object Adapter avrà
  - (a) Solo una istanza per il ruolo Adapter
  - (b) Una istanza per ciascun ruolo Adapter, Adaptee, Target
  - (c) Una istanza per ciascun ruolo Adapter e Adaptee
  - (d) Nessuna istanza
8. Un vantaggio offerto dal design pattern Facade è
  - (a) Ridurre le dipendenze fra client e Facade
  - (b) Ridurre il numero di classi
  - (c) Ridurre le dipendenze fra Facade e classi del sottosistema
  - (d) Ridurre le dipendenze fra client e classi del sottosistema
9. Nel processo XP
  - (a) I test sono scritti prima o durante l'implementazione delle funzionalità
  - (b) E' raccomandato usare l'UML
  - (c) Il cliente collabora durante l'ispezione del codice
  - (d) È suggerito lavorare 48 ore la settimana
10. Per il design pattern Composite
  - (a) Si ha sicurezza quando si autentica l'utente
  - (b) Si possono ottenere trasparenza e sicurezza nella stessa versione
  - (c) Si ha trasparenza quando il client distingue oggetti semplici e composti
  - (d) La versione che fornisce trasparenza non può fornire sicurezza
11. La classe C contiene i metodi m1() e m2(), quando m1() chiama m2()
  - (a) Si esegue il metodo di C o quello di una sua superclasse
  - (b) Si esegue sicuramente il metodo di C
  - (c) Si esegue il metodo che il compilatore ha individuato
  - (d) Si esegue il metodo di C o quello di una sua sottoclasse
12. Quale delle seguenti non è una caratteristica del pattern Prototype?
  - (a) Se clono un prototipo in modalità shallow le copie potrebbero interferire l'una con l'altra
  - (b) Il prototipo è istanziato esternamente e poi passato come parametro
  - (c) Ho una potenziale perdita di efficienza per l'aumento del numero delle classi
  - (d) Quando la creazione di nuove istanze è costosa, clonando miglioro la performance
13. Per il design pattern Decorator, il ruolo Decorator
  - (a) Tiene un riferimento
  - (b) E' una classe astratta
  - (c) Può essere un'interfaccia
  - (d) E' un'interfaccia
14. Nel design pattern State, l'istanza di un sottotipo di State
  - (a) Cambia classe, all'occorrenza
  - (b) Non è conosciuta dalle classi client
  - (c) È riferita dal tipo State
  - (d) È creata da classi client
15. Il design pattern Bridge permette
  - (a) Di togliere una parte di implementazione ad una certa classe
  - (b) Alle classi client di conoscere quanti metodi verranno eseguiti sulla piattaforma
  - (c) Di togliere funzionalità ad una certa istanza a runtime
  - (d) Alle classi client di distinguere fra varie astrazioni

16. Per il design pattern Chain of Responsibility
- (a) Le classi richiedenti devono essere più di una
  - (b) I possibili riceventi sono più di uno
  - (c) Si richiede di implementare tanti metodi per ciascuna classe ricevente
  - (d) Si ha che il numero di richiedenti è pari al numero di riceventi

Nome

Matr.

1. Nei diagrammi UML degli stati, i nomi degli stati sono indicati
  - (a) All'interno di rettangoli senza angoli arrotondati
  - (b) All'interno di rettangoli con angoli arrotondati
  - (c) Sopra i rettangoli con angoli arrotondati
  - (d) Alle estremità delle frecce delle transizioni
2. Il design pattern Observer è consigliabile quando
  - (a) Si vuol separare bene il codice di due classi
  - (b) Si vuol chiamare un metodo di un'altra classe
  - (c) Si vuol tenere coerente lo stato di una classe con quello di un'altra classe
  - (d) Si vuol tenere separato lo stato di due oggetti
3. Il design pattern Decorator permette
  - (a) Di aggiungere o togliere un piccolo compito ad una classe
  - (b) Solo di aggiungere un piccolo compito ad una classe
  - (c) Di togliere un metodo ad una classe
  - (d) Di ridefinire un metodo di una classe
4. Il design pattern Bridge permette alla classe client di
  - (a) Tenere alcuni riferimenti a oggetti sottotipi di Implementor
  - (b) Non tenere riferimenti
  - (c) Non dover chiamare i metodi di Implementor
  - (d) Tenere riferimenti a oggetti che sono sottotipi di Implementor
5. La classe che ha il ruolo Facade ha
  - (a) Una sola classe nel sottosistema
  - (b) Nessuna classe client, difatti il Facade potrebbe avere il metodo main
  - (c) Una sola classe client
  - (d) Un certo numero di classi client
6. Tipicamente per il processo XP, le release del software avvengono
  - (a) Una release alla settimana
  - (b) Una release ogni 2 settimane
  - (c) Una release ogni 3 settimane
  - (d) Una release ogni 4 settimane
7. La fase di progettazione produce e documenta
  - (a) La struttura del software che realizza le specifiche
  - (b) Le specifiche del software
  - (c) Le modalità di utilizzo del software
  - (d) Le funzionalità del software
8. Il design pattern Adapter serve a
  - (a) Convertire una interfaccia di una classe in un'altra
  - (b) Convertire il tipo di una variabile
  - (c) Cambiare l'interfaccia di una classe a runtime
  - (d) Usare due nomi per una stessa classe
9. Si abbia il frammento di codice:

```
public Libro getLibro() {  
    return new Book();  
}
```

  - (a) Il frammento sta svolgendo il ruolo di un Factory Method
  - (b) Il frammento sta svolgendo il ruolo di un Singleton
  - (c) Il codice non è compilabile
  - (d) Il codice dovrebbe essere all'interno di una superclasse
10. Per il design pattern Chain of Responsibility
  - (a) I possibili riceventi sono più di uno
  - (b) Le classi richiedenti devono essere più di una
  - (c) Si ha che il numero di richiedenti è pari al numero di riceventi
  - (d) Si richiede di implementare tanti metodi per ciascuna classe ricevente
11. Quale delle seguenti è uno svantaggio dell'uso del pattern Command?
  - (a) Gli stessi comandi potrebbero essere generati da elementi diversi dell'interfaccia
  - (b) Si deve usare il pattern Singleton per imporre un gestore unico dei comandi
  - (c) Si potrebbero penalizzare le prestazioni a causa dell'indirizzione introdotta
  - (d) I comandi ancora non eseguiti potrebbero essere anche annullati
12. I design pattern sono strutture
  - (a) Create appositamente quando si risolve un nuovo problema
  - (b) Per un piccolo numero di classi
  - (c) Utili alla fase di raccolta ed analisi dei requisiti
  - (d) Per un grande numero di classi
13. Quale legge è soddisfatta quando si usa il Mediator?
  - (a) La legge di Murphy
  - (b) La prima legge di Lehman
  - (c) La legge di Demeter
  - (d) La legge sui grandi numeri
14. Per un sistema che è Spaghetti code
  - (a) Il codice è stato creato insieme al cliente
  - (b) Si hanno classi con tanti metodi
  - (c) E' stata seguita la progettazione ad oggetti
  - (d) Non è possibile far test
15. Per il design pattern Composite, si ha trasparenza quando il client
  - (a) Distingue oggetti semplici e composti
  - (b) Può usare oggetti semplici o composti
  - (c) Non deve distinguere fra oggetti semplici e composti
  - (d) Non può usare oggetti semplici o composti

16. Per eliminare le istruzioni condizionali

- (a) Si possono inserire le precondizioni
- (b) Si potrebbe usare il design pattern Facade
- (c) Si potrebbe usare il design pattern Singleton
- (d) Si potrebbe usare il design pattern State